



ENTERPRISE ANALYTICS HAKATHON REPORT

Executive Summary



MEHREEN FATIMA

Executive Summary

Modern enterprise databases are primarily designed to support transactional operations such as order processing, inventory management, customer management, and purchasing. While these databases efficiently handle day-to-day business operations, they are not optimized for analytical reporting because business metrics often require repeated joins, aggregations, and calculations across multiple tables.

The objective of this project was to design and implement a reusable analytics layer on top of an enterprise PostgreSQL database. Instead of querying raw operational tables repeatedly, analytical views were developed to centralize business logic and create reusable datasets for reporting. These views were then consumed in a Python-based analytics notebook to generate executive dashboards and business insights.

The solution follows the principles of Analytics Engineering by separating operational data from analytical reporting, ensuring scalability, maintainability, and improved query performance.

2. Project Objectives

The project was designed to achieve the following objectives:

- Build a reusable analytics layer over transactional data.
- Minimize repeated querying of operational tables.
- Create analytical views for multiple business domains.
- Develop a chained SQL pipeline for reusable reporting.
- Generate executive KPI datasets.
- Create business dashboards using Python.
- Produce actionable business recommendations.

3. Problem Statement

The operational AdventureWorks database contains highly normalized transactional tables optimized for data integrity and transactional processing. However, directly querying these tables for reporting introduces several challenges:

- Complex SQL joins
- Repeated calculations

- Poor maintainability
- Difficult dashboard development
- High query complexity

To address these issues, an intermediate analytics layer was developed that transforms raw operational data into reusable analytical datasets.

4. Project Overview

The project consists of two major components:

SQL Analytics Layer

Reusable analytical views were created to preprocess and organize business information.

Python Executive Dashboard

A Jupyter Notebook was developed to read only the analytical views and generate executive dashboards with business insights.

5. Database Overview

The project uses the AdventureWorks enterprise database containing business information from multiple operational departments.

Major schemas used include:

- Sales
- Production
- Purchasing
- Human Resources
- Person
- Inventory

The database contains information about:

- Customers
- Orders

- Products
- Employees
- Vendors
- Inventory
- Territories

6. Analytics Architecture

The project follows a layered analytics architecture.

Raw Operational Tables

|



Analytics Base Views

|



Business Analytics Views

|



Executive KPI Views

|



Python Dashboard

This architecture ensures that every dashboard reads only analytical datasets instead of querying raw transactional tables.

7. Business Domains Covered

The analytics layer was developed across multiple business domains.

Business Domain	Description
Sales	Revenue and sales performance
Customers	Customer behavior and segmentation
Products	Product profitability and rankings
Employees	Employee sales performance
Territories	Regional revenue analysis
Inventory	Inventory health monitoring
Vendors	Purchasing and supplier analysis

The project satisfies the requirement of using more than five business domains.

8. SQL Pipeline Design

The SQL implementation follows a chained dependency structure.

Operational Tables

|



Base Analytics Views

|



Business Metrics

|



Customer Segmentation

|



Regional Analysis

|



Executive KPIs

|



Python Visualizations

Each stage reuses previously created views instead of recalculating business metrics.

9. Intermediate Analytics Views

The following reusable analytical views were created.

Base Views

- vw_sales_base
- vw_customer_base
- vw_product_base
- vw_employee_base
- vw_inventory_base
- vw_vendor_base
- vw_territory_base
- vw_date_base
- vw_executive_base

Business Analytics Views

- vw_monthly_revenue
- vw_sales_growth
- vw_customer_segments
- vw_customer_lifetime_value
- vw_customer_retention

- vw_product_performance
- vw_product_profitability
- vw_product_ranking
- vw_employee_performance
- vw_employee_ranking
- vw_regional_revenue
- vw_regional_growth
- vw_inventory_health
- vw_supplier_performance
- vw_purchasing_trends

Executive KPI Views

- vw_best_selling_products
- vw_repeat_customers
- vw_revenue_contribution
- vw_performance_comparison

These analytical views eliminate repeated SQL calculations and simplify dashboard development.

10. Advanced SQL Concepts Implemented

The project demonstrates several advanced SQL techniques.

Multiple Chained CTEs

Used for:

- Customer Retention
- Regional Growth
- Sales Growth

Window Functions

Implemented using:

- RANK()
- DENSE_RANK()
- ROW_NUMBER()
- LAG()

CASE WHEN

Used for:

- Customer Segmentation
- Inventory Classification
- Employee Performance Categories

Conditional Aggregation

Used for:

- Revenue calculations
- Inventory summaries
- Product profitability

Ranking Functions

Used for:

- Product Ranking
- Employee Ranking
- Territory Ranking

Complex JOINS

Multiple business domains were combined using joins involving:

- Sales
- Products
- Customers
- Employees
- Vendors
- Territories

11. Executive KPI Datasets

The analytics layer generates dashboard-ready datasets.

Sales KPIs

- Monthly Revenue
- Sales Growth
- Best Selling Products

Customer KPIs

- Customer Segments
- Customer Lifetime Value
- Repeat Customers
- Customer Retention

Product KPIs

- Product Performance
- Product Profitability
- Product Rankings

Employee KPIs

- Employee Performance
- Revenue Contribution
- Performance Comparison

Territory KPIs

- Regional Revenue
- Regional Growth
- Top Territories

Inventory KPIs

- Inventory Health
- Low Stock Products

Purchasing KPIs

- Supplier Performance
- Purchasing Trends

12. Python Analytics Dashboard

The analytical views were connected to PostgreSQL using SQLAlchemy and Pandas.

The notebook reads only the analytical views.

Visualizations created include:

- Monthly Revenue Trend
- Sales Growth
- Revenue by Territory
- Customer Segmentation
- Product Performance
- Employee Performance
- Inventory Health
- Best Selling Products

Each visualization includes a business explanation describing management insights.

13. Business Insights

Revenue Analysis

Monthly revenue analysis identifies seasonal fluctuations and assists management in planning promotional campaigns during low-performing periods.

Territory Performance

A small number of territories generate the majority of company revenue, indicating opportunities for geographic expansion and targeted marketing.

Customer Behavior

VIP and repeat customers contribute significantly to total revenue, emphasizing the importance of customer retention strategies.

Product Performance

Best-selling products consistently drive business revenue. Maintaining inventory availability for these products is essential.

Employee Performance

Employee rankings highlight performance differences that can support incentive programs and training initiatives.

Inventory Analysis

Inventory health monitoring identifies products approaching critical stock levels before stockouts occur.

14. Executive Recommendations

Business Opportunities

1. Expand investment in top-performing territories.
2. Increase inventory for best-selling products.
3. Improve customer loyalty programs.
4. Promote highly profitable products.
5. Replicate successful sales practices of top-performing employees.

Business Risks

1. Poor-performing territories reduce growth potential.
2. Critical inventory shortages may interrupt sales.
3. Revenue dependency on a limited number of products.
4. Customer retention decline may reduce future revenue.
5. Uneven employee performance impacts sales consistency.

Actionable Recommendations

1. Implement automated inventory monitoring.
2. Develop customer retention campaigns.
3. Increase marketing for profitable products.
4. Review sales KPIs monthly.

5. Conduct regular employee performance evaluations.

15. Challenges Faced

Several implementation challenges were encountered during the project.

- PostgreSQL database import issues due to AdventureWorks CSV formatting.
- Schema compatibility differences between AdventureWorks versions.
- Missing calculated columns such as LineTotal in some tables.
- Data import inconsistencies requiring adjustments to SQL scripts.
- Standardizing analytical view names across the project.
- Resolving connection issues between PostgreSQL and Python.

These challenges were resolved by validating table structures, modifying SQL queries, and creating reusable analytical views.

16. Assumptions

The following assumptions were made during development:

- Operational data was assumed to be accurate after import.
- Missing calculated fields were derived during SQL transformations.
- Analytics views were treated as the single source of truth for reporting.
- Python dashboards accessed only analytical views and never raw transactional tables.
- Business metrics were calculated using available transactional data.

17. Future Improvements

Future enhancements may include:

- Materialized views for improved performance.
- Automated ETL pipelines.
- Incremental data refresh.
- Power BI or Tableau integration.

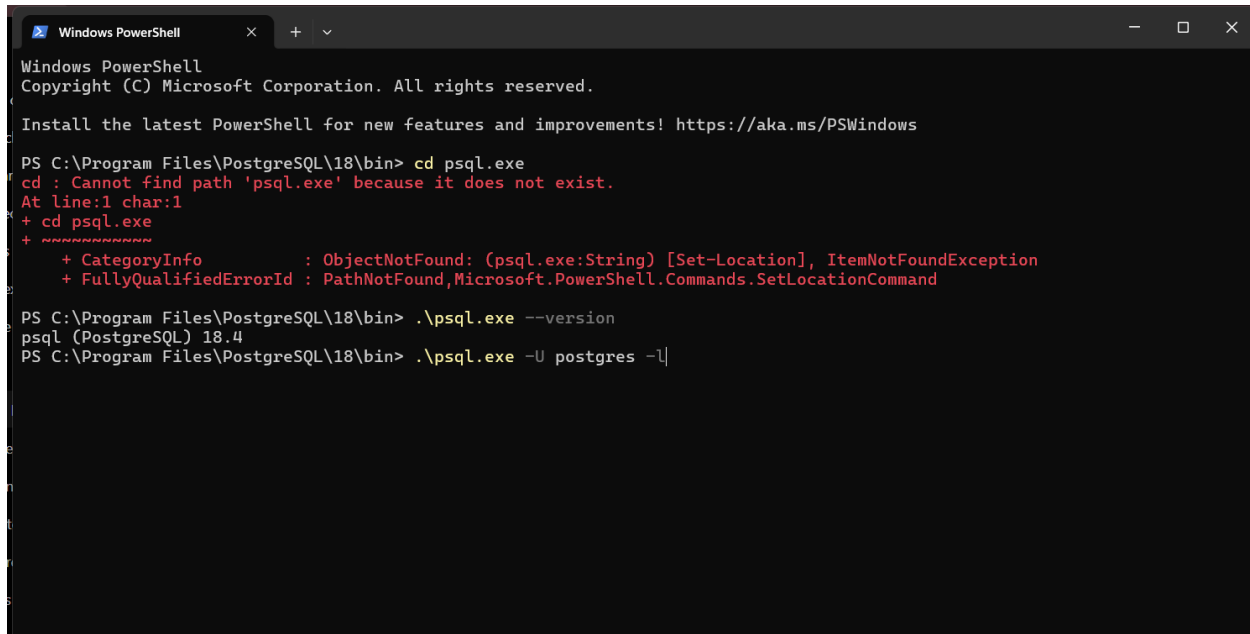
- Machine learning-based sales forecasting.
- Customer churn prediction models.
- Inventory demand forecasting.
- Interactive web dashboards using Streamlit or Dash.

18. Conclusion

This project successfully demonstrates the implementation of a reusable enterprise analytics layer using PostgreSQL and Python.

Instead of repeatedly querying normalized operational tables, reusable analytical views were created to centralize business logic and simplify reporting. The SQL pipeline supports future dashboard development while maintaining scalability and maintainability.

The Python notebook consumes only analytical views to generate executive dashboards that provide valuable business insights. The overall solution follows Analytics Engineering principles and establishes a robust foundation for future business intelligence and reporting initiatives.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Program Files\PostgreSQL\18\bin> cd psql.exe
cd : Cannot find path 'psql.exe' because it does not exist.
At line:1 char:1
+ cd psql.exe
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (psql.exe:String) [Set-Location], ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

PS C:\Program Files\PostgreSQL\18\bin> .\psql.exe --version
psql (PostgreSQL) 18.4
PS C:\Program Files\PostgreSQL\18\bin> .\psql.exe -U postgres -l
```

Database creation

```
Windows PowerShell
psql (PostgreSQL) 18.4
PS C:\Program Files\PostgreSQL\18\bin> .\psql.exe -U postgres -l
Password for user postgres:

      Name      | Owner | Encoding | Locale Provider |      List of databases      |
  Locale | ICU Rules | Access privileges | Collate | Ctype
-----+-----+-----+-----+-----+-----+
  dvdrental      | postgres | UTF8      | libc            | English_United States.1252 | English_United States.1252
  enterprise_analytics | postgres | UTF8      | libc            | English_United States.1252 | English_United States.1252
  music_store     | postgres | UTF8      | libc            | English_United States.1252 | English_United States.1252
  postgres       | postgres | UTF8      | libc            | English_United States.1252 | English_United States.1252
  template0      | postgres | UTF8      | libc            | English_United States.1252 | English_United States.1252
  |              | =c/postgres | +              | | |
  |              | postgres=CtC/postgres | | |
  template1      | postgres | UTF8      | libc            | English_United States.1252 | English_United States.1252
  |              | =c/postgres | +              | | |
  |              | postgres=CtC/postgres | | |
(6 rows)
```

Reference files:

```
Windows PowerShell
PS C:\Program Files\PostgreSQL\18\bin> cd "C:\Users\as\Downloads\week3_hackathon_dataset\week3_hackathon_dataset"
PS C:\Users\as\Downloads\week3_hackathon_dataset\week3_hackathon_dataset> dir

Directory: C:\Users\as\Downloads\week3_hackathon_dataset\week3_hackathon_dataset

Mode                LastWriteTime         Length Name
----                -
-a-----          7/10/2026 11:13 AM      2942518 Address.csv
-a-----          7/10/2026 11:13 AM         428 AddressType.csv
-a-----          7/10/2026 11:13 AM          62 AWBuildVersion.csv
-a-----          7/10/2026 11:13 AM     198923 BillOfMaterials.csv
-a-----          7/10/2026 11:13 AM    1464061 BusinessEntity.csv
-a-----          7/10/2026 11:13 AM    1576943 BusinessEntityAddress.csv
-a-----          7/10/2026 11:13 AM     71972 BusinessEntityContact.csv
-a-----          7/10/2026 11:13 AM         946 ContactType.csv
-a-----          7/10/2026 11:13 AM         9066 CountryRegion.csv
-a-----          7/10/2026 11:13 AM        3379 CountryRegionCurrency.csv
-a-----          7/10/2026 11:13 AM    1203673 CreditCard.csv
-a-----          7/10/2026 11:13 AM         337 Culture.csv
-a-----          7/10/2026 11:13 AM        4312 Currency.csv
-a-----          7/10/2026 11:13 AM    1022949 CurrencyRate.csv
-a-----          7/10/2026 11:13 AM    1717324 Customer.csv
-a-----          7/10/2026 11:13 AM        1075 Department.csv
-a-----          7/10/2026 11:13 AM     521424 Document.csv
-a-----          7/10/2026 11:13 AM    2127590 EmailAddress.csv
-a-----          7/10/2026 11:13 AM     48854 Employee.csv
-a-----          7/10/2026 11:13 AM     13021 EmployeeDepartmentHistory.csv
-a-----          7/10/2026 11:13 AM     19425 EmployeePayHistory.csv
```

Imported tables

```
Windows PowerShell
```

List of tables			
Schema	Name	Type	Owner
humanresources	department	table	postgres
humanresources	employee	table	postgres
humanresources	employeedepartmenthistory	table	postgres
humanresources	employeeepayhistory	table	postgres
humanresources	jobcandidate	table	postgres
humanresources	shift	table	postgres
person	address	table	postgres
person	addressstype	table	postgres
person	businessentity	table	postgres
person	businessentityaddress	table	postgres
person	businessentitycontact	table	postgres
person	contacttype	table	postgres
person	countryregion	table	postgres
person	emailaddress	table	postgres
person	password	table	postgres
person	person	table	postgres
person	personphone	table	postgres
person	phonenumbertype	table	postgres
person	stateprovince	table	postgres
production	billofmaterials	table	postgres
production	culture	table	postgres
production	document	table	postgres
production	illustration	table	postgres
production	location	table	postgres
production	product	table	postgres
production	productcategory	table	postgres

```
-- More --
```

Schema Creation

Query

Query History

```
1  /*****
2  ENTERPRISE ANALYTICS HACKATHON
3  Stage 1 - Base Analytics Layer
4
5  Author : Mehreen Fatima
6  Database : AdventureWorks
7  *****/
8
9  -----
10 -- Create Analytics Schema
11 -----
12
13 CREATE SCHEMA IF NOT EXISTS analytics;
14
```

Data Output

Messages

Notifications

```
CREATE SCHEMA
Query returned successfully in 78 msec.
```

✓ Query returned successfully in 78 msec. ✕

Total rows: Query complete 00:00:00.078 CRLF Ln 14, Col 1

Column Names:

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```

36
37     ROUND(
38         sod.orderqty * sod.unitprice * (1 - sod.unitpricediscount),
39         2
40     ) AS line_total,
41
42     soh.subtotal,
43     soh.taxamt,
44     soh.freight,
45     soh.totaldue
46
47 FROM sales.salesorderheader soh
48 JOIN sales.salesorderdetail sod
49 ON soh.salesorderid = sod.salesorderid;
50

```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 107 msec.

✓ Query returned successfully in 107 msec. ✕

Total rows: Query complete 00:00:00.107 CRLF Ln 49, Col 40

Customer Base view:

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```

63     p.businessname,
64
65     CONCAT(p.firstname, ' ', p.lastname) AS customer_name,
66
67     s.name AS store_name
68
69 FROM sales.customer c
70
71 LEFT JOIN person.person p
72
73 ON c.personid = p.businessentityid
74
75 LEFT JOIN sales.store s
76
77 ON c.storeid = s.businessentityid;

```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 66 msec.

✓ Query returned successfully in 66 msec. ✕

Total rows: Query complete 00:00:00.066 CRLF Ln 77, Col 35

Product Base view

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```

100     sc.name AS subcategory,
101
102     pc.productcategoryid,
103
104     pc.name AS category
105
106 FROM production.product p
107
108 LEFT JOIN production.productssubcategory sc
109
110 ON p.productssubcategoryid = sc.productssubcategoryid
111
112 LEFT JOIN production.productcategory pc
113
114 ON sc.productcategoryid = pc.productcategoryid;

```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 91 msec.

✓ Query returned successfully in 91 msec. ✕

Total rows: Query complete 00:00:00.091 CRLF Ln 114, Col 48

Employee Base View

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```

125     sp.salesquotaytd,
126
127     sp.bonus,
128
129     sp.commissionpct,
130
131     sp.salesytd,
132
133     sp.saleslastyear
134
135 FROM sales.salesperson sp
136
137 JOIN person.person pp
138
139 ON sp.businessentityid = pp.businessentityid;

```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 82 msec.

✓ Query returned successfully in 82 msec. ✕

Total rows: Query complete 00:00:00.082 CRLF Ln 139, Col 46

Territory Base View

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```
139 ON sp.businessentityid = pp.businessentityid,  
140 --Territory base view  
141 CREATE OR REPLACE VIEW analytics.vw_territory_base AS  
142  
143 SELECT  
144     territoryid,  
145     name AS territory,  
146     countryregioncode,  
147     "group"  
148  
149 FROM sales.salesterritory;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 103 msec.

Microsoft Store

✓ Query returned successfully in 103 msec. ✕

Total rows: Query complete 00:00:00.103 CRLF Ln 153, Col 27

Inventory Base View

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```
161 p.name AS product_name,  
162  
163 pi.locationid,  
164  
165 pi.quantity,  
166  
167 pi.shelf,  
168  
169 pi.bin  
170  
171 FROM production.productinventory pi  
172  
173 JOIN production.product p  
174  
175 ON pi.productid = p.productid;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 98 msec.

Microsoft Store

✓ Query returned successfully in 98 msec. ✕

Total rows: Query complete 00:00:00.098 CRLF Ln 175, Col 31

Vendor Base View

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```
174
175 ON pi.productid = p.productid;
176
177 --Vendor Base View
178 CREATE OR REPLACE VIEW analytics.vw_vendor_base AS
179
180 SELECT
181     v.businessentityid,
182     v.accountnumber,
183     v.name,
184     v.creditrating
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 160 msec.

✓ Query returned successfully in 160 msec. ✕

Total rows: Query complete 00:00:00.163 CRLF Ln 178, Col 1

Data Analytics Base View:

enterprise_analytics_new/postgres@PostgreSQL 18

Query Query History

```
205 SELECT DISTINCT
206
207     orderdate,
208
209     EXTRACT(YEAR FROM orderdate) AS year,
210
211     EXTRACT(QUARTER FROM orderdate) AS quarter,
212
213     EXTRACT(MONTH FROM orderdate) AS month,
214
215     TO_CHAR(orderdate, 'Month') AS month_name,
216
217     TO_CHAR(orderdate, 'Day') AS day_name
218
219 FROM sales.salesorderheader;
```

Data Output Messages Notifications

CREATE VIEW

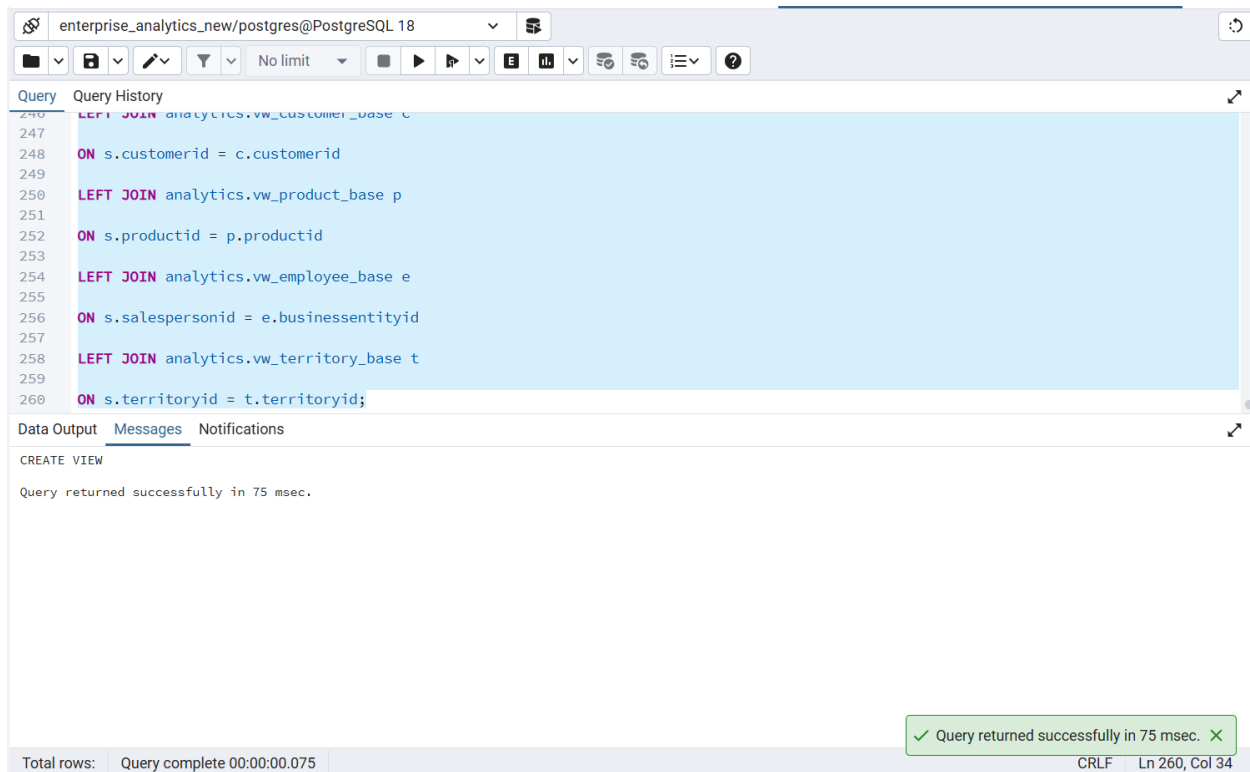
Query returned successfully in 126 msec.

✓ Query returned successfully in 126 msec. ✕

Total rows: Query complete 00:00:00.126 CRLF Ln 219, Col 29

Executive Base View

This is the first reusable business layer.

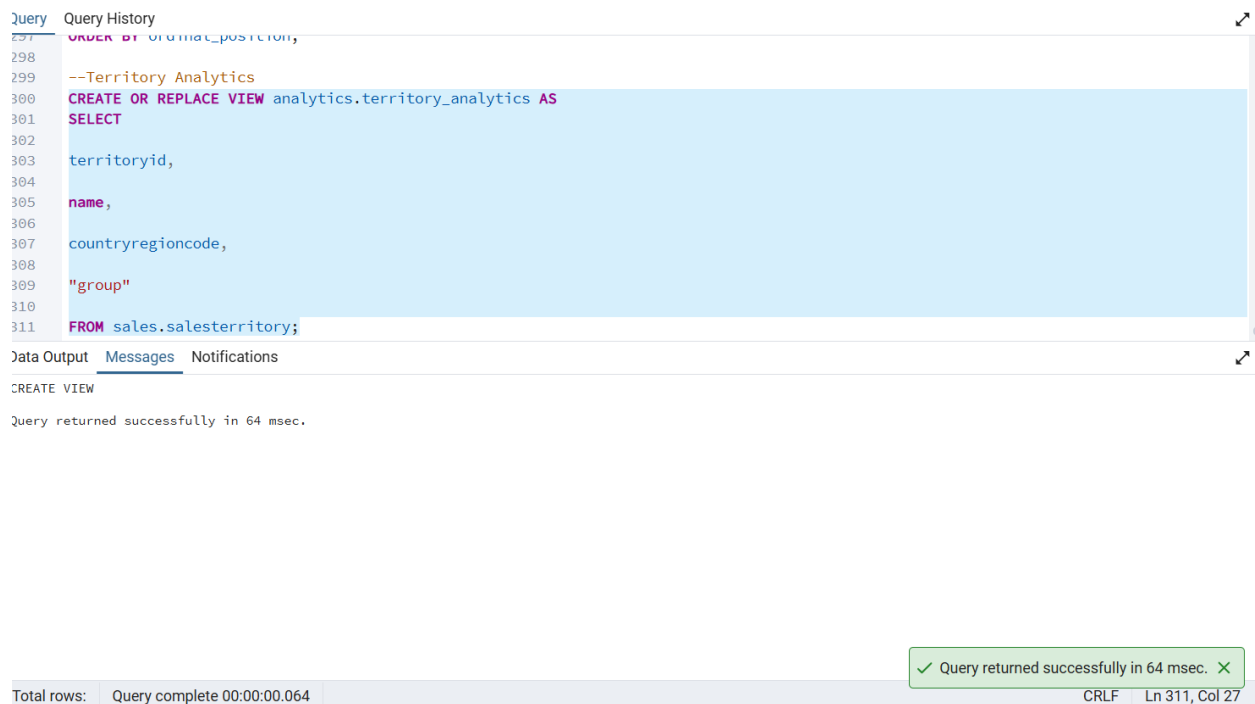


The screenshot shows a PostgreSQL query editor window titled "enterprise_analytics_new/postgres@PostgreSQL 18". The query is a complex JOIN statement:

```
246 LEFT JOIN analytics.vw_customer_base c
247
248 ON s.customerid = c.customerid
249
250 LEFT JOIN analytics.vw_product_base p
251
252 ON s.productid = p.productid
253
254 LEFT JOIN analytics.vw_employee_base e
255
256 ON s.salespersonid = e.businessentityid
257
258 LEFT JOIN analytics.vw_territory_base t
259
260 ON s.territoryid = t.territoryid;
```

The "Data Output" tab shows the message: "CREATE VIEW" and "Query returned successfully in 75 msec." A green status bar at the bottom right indicates: "✓ Query returned successfully in 75 msec. ✕". The bottom status bar shows "Total rows: Query complete 00:00:00.075" and "CRLF Ln 260, Col 34".

Territory Analytics:



The screenshot shows a PostgreSQL query editor window. The query is a view creation statement:

```
297 ORDER BY ordinal_position,
298
299 --Territory Analytics
300 CREATE OR REPLACE VIEW analytics.territory_analytics AS
301 SELECT
302
303 territoryid,
304
305 name,
306
307 countryregioncode,
308
309 "group"
310
311 FROM sales.salesterritory;
```

The "Data Output" tab shows the message: "CREATE VIEW" and "Query returned successfully in 64 msec." A green status bar at the bottom right indicates: "✓ Query returned successfully in 64 msec. ✕". The bottom status bar shows "Total rows: Query complete 00:00:00.064" and "CRLF Ln 311, Col 27".

Purchasing Base Analytics:

enterprise_analytics_new/postgres@PostgreSQL 18

Query History

```

330 pod.orderqty,
331
332 pod.unitprice,
333
334 pod.linetotal
335
336 FROM purchasing.purchaseorderheader poh
337
338 JOIN purchasing.purchaseorderdetail pod
339
340 ON poh.purchaseorderid=pod.purchaseorderid
341
342 JOIN analytics.vendor_analytics v
343
344 ON poh.vendordid=v.businessentityid;
  
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 82 msec.

Total rows: Query complete 00:00:00.082 CRLF Ln 344, Col 36

Task 02:

Dependency Chain

Raw Tables



Task 1 Analytics Views



Business Metrics



Customer Segmentation



Regional Analysis



Executive KPI Views



Python Dashboard

This satisfies your supervisor's requirement.